Name: Pathri Vidya Praveen

Roll No.: CS24BTECH11047

Course: CS3563 Introduction to DBMS II

Homework 1

1. ⇒ A mailing application, mess bill statistics application performance analysis application are running on IITH student DBMS.

⇒ Physical layout of student records is:

Roll No. | Name | Address | GPA | Hostel Name | Mess bill

⇒ To improve operational efficiency, structure of student records is changed such that all numeric fields appear in beginning, all character fields appear in the end.

GPA | Mess bill | Roll No. | Name | Address | Hostel Name

(numeric) (character)

⇒ We don't need to make any changes as these applications use relational algebra or SQL queries. Applications are independent of physical layout structure. DBMS internally maps logical layer to physical layer maintaining physical data independence. Attributes and their data are not changed — they are just reordered.

⇒ No modifications required as the offsets need not be changed by us manually because of internal mapping by DBMS

2.

Table 1

Name	Year	Revenue
Bahubali	2015	600
Pushpa	2021	350
Dhurandhar	2025	1000

Table 2

Year	Name	Revenue
2015	Bahubali	600
2021	Pushpa	350
2025	Dhurandhar	1000
2015	Bahubali	600

⇒ Table 1 has 3 distinct tuples with no duplicates. But Table 2 has 4 tuples including a duplicate (2015, Bahubali, 600). In the relational model, relation is set of tuples which are unordered. Table 2 is violating this so not valid relation

⇒ Table 1 has attributes in order Name–Year–Revenue. Table 2 has attributes in order Year–name–Revenue. Attribute order does not matter in relational model as relations are defined by attribute names not positions.

3. ⇒ SQL query on student database:

```
select T.cname from Course T, Course S
where T.credits > S.credits and
S.dept = "CSE"
```

⇒ Basically this query is trying to output all course names that have more no. of credits than atleast 1 course from CSE department.

⇒ 6 fundamental operators — ① σ (select) ② π (project) ③ ρ (rename) ④ × (cartesian product) ⑤ ∪ (union), ⑥ − (set difference).

⇒ ① Select CSE department courses

$\sigma_{dept = "CSE"}(Course)$

② Use only credits of these CSE courses, rename

$\rho_{cse\_credits \leftarrow credits}(\pi_{credits}(\sigma_{dept = "CSE"}(Course)))$

③ Do cartesian product and select pairs where course credits exceed CSE course credits.

$\sigma_{credits > cse\_credits}(Course \times \rho_{cse\_credits \leftarrow credits}(\pi_{credits}(\sigma_{dept = "CSE"}(Course))))$

④ Finally project only course names

⇒ $\pi_{cname}(\sigma_{credits > cse\_credits}(Course \times \rho_{cse\_credits \leftarrow credits}(\pi_{credits}(\sigma_{dept = "CSE"}(Course)))))$

⇒ Here, we apply $\sigma_{dept = "CSE"}$ immediately to reduce CSE courses relation before cartesian product. We only keep credits attribute from CSE courses, reducing size of relations. We also avoid comparing duplicates. So, this is the most efficient query.

⇒ One more approach naively is

$$\Pi_{T.cname}\left(\sigma_{T.credits > S.credits \wedge S.dept = \text{"CSE"}}\left(\rho_T(Course) \times \rho_S(Course)\right)\right)$$

↳ direct translation of SQL query.

This is less efficient because it computes full cartesian product of Course x Course and then applies dept = "CSE" filter after this product which is unnecessary.

4. ⇒ LOJ = Left Outer Join

⇒ LOJ for R and S is defined to be the union of (a) the natural join of R and S with (b) set of tuples from R that do not join with any tuples in S, each concatenated with a 'null' S tuple (i.e S tuple composed of all null fields)

(1.) Yes, there is a relational algebra equivalent for LOJ using fundamental operators.

$$R \text{ LOJ } S = (R \bowtie S) \cup (R - \pi_R(R \bowtie S)) \times \{(null, null, \ldots, null)\}$$

(2.) ⇒ LOJ is not associative. Proof by counter example

⇒ R(X, Y), S(X, Z), T(Z, 4)

$R = \{(1, 10)\}$, $S = \{(1, 20)\}$, $T = \{(20, 30)\}$

⇒ $R \text{ LOJ } S = \{(1, 10, 20)\}$

$(R \text{ LOJ } S) \text{ LOJ } T = \{(1, 10, 20)\}$

⇒ $S \text{ LOJ } T = \{(1, 20, 30)\}$

$R \text{ LOJ } (S \text{ LOJ } T) = \{(1, 10, null)\}$

⇒ $(R \text{ LOJ } S) \text{ LOJ } T \neq R \text{ LOJ } (S \text{ LOJ } T)$

(3.) ⇒ LOJ is not commutative.

⇒ Assume LOJ is commutative and give counter example for this.

⇒ $R = \{(1)\}$, $S = \{(2)\}$

⇒ R LOJ $S = \{(1, null)\}$, S LOJ $R = \{(2, null)\}$

⇒ R LOJ $S \neq S$ LOJ R

⇒ In R LOJ S, all tuples from R are preserved but in S LOJ R, all tuples from S are preserved. So if R, S have non-matching tuples, we get different results.

(4.) ⇒ Min. cardinality of R LOJ $S = |R|$.

⇒ In LOJ every tuple from R must appear in result. If any tuple does not match any tuple in S, it appears exactly once padded with nulls. If tuple matches exactly 1 tuple in S, it appears exactly once.

⇒ Cardinality ↑ if any tuple in R matches multiple tuples in S.

⇒ $|R$ LOJ $S| \geq |R|$.

---

5. ⇒ Database schema:

COMPANY (cname, type)

WORKER (EBno, wname, wstate)

INTERVIEW (cname, EBno, date)

PROJECT (pname, cname, pstate)

EXPERIENCE (EBno, cname, years)

HIRE (cname, pname, EBno, salary)

⇒ HIRE — info about current employees hired by companies,

⇒ EXPERIENCE — stores info about past employment of employees

⇒ It is possible to write SQL expression.

⇒ SQL query:

```
WITH ValidCompanyTypes AS (
    SELECT p.cname FROM PROJECT p
    GROUP BY p.cname
    HAVING COUNT(DISTINCT pstate) = 1
    OR COUNT(DISTINCT pstate) =
                    COUNT(pname)
),   --Fully centralized or fully-distributed
CompaniesWithBadHires AS (
    SELECT DISTINCT h.cname
    FROM HIRE h, JOIN PROJECT p ON
    h.cname = p.cname AND h.pname=p.pname
    JOIN WORKER w ON h.EBno = w.EBno
    WHERE p.pstate <> w.wstate
    OR (SELECT COUNT(*) FROM
    EXPERIENCE e WHERE e.EBno = w.EBno)
        <> 3
)
-- private companies that are valid and not hire bad
SELECT c.cname FROM COMPANY c JOIN
ValidCompanyTypes v ON c.cname = v.cname
where c.type = 'private' and c.cname NOT IN (SELECT
cname from CompaniesWithBadHires) ORDER BY c.cname
                                    ASC;
```

⇒ 'Valid Company Types' identifies companies either having all projects in 1 state (centralized) or each project in unique state (distributed)

⇒ 'Companies With Bad Hires' is companies that violate rules and employ non-local workers or workers without exactly 3 past jobs.

⇒ Query filters COMPANY table for private types, perform inner join with Valid Company Types to ensure only 2 specific categories considered.

⇒ Company is only select if all its current employees meet local, experience criteria.

⇒ ORDER BY — lexicographical order.